



**INSTITUTO DE EDUCACIÓN SECUNDARIA LA MOLA
NOVELDA**

DESARROLLO DE APLICACIONES WEB

Curso Académico 2025/2026

Desarrollo Web en Entorno Cliente

**U04_A02 Casos prácticos, Ponte a prueba, etc... -
Prof. Emilio Ramírez**

Autor: Segura Abad, Mario

Fecha: 11 de Enero, 2026

INTRODUCCIÓN 1

ENUNCIADO 2

 Probad todo el código del Apartado 2 3

 Probad los Scripts del Apartado 2 3

 Caso práctico 2 3

 Ponte a prueba 2 3

 Claves de resolución:..... 3

DESARROLLO DE LA PRACTICA 4

 Descripción 5

 Contenido del archivo 5

 Código del Apartado 2 de teoría 5

 • Creación de promesas mediante new Promise(). 5

 • Resolución y rechazo de promesas con resolve y reject..... 5

 • Encadenamiento de tareas usando .then(). 5

 • Manejo de errores con .catch(). 5

 • Ejecución de código final con .finally(). 5

 Caso Práctico 2 – Encadenamiento de promesas 5

 Objetivo:..... 5

 Claves de resolución:..... 5

 • Crear una función temporizador(t) que devuelva una promesa que se resuelve tras t milisegundos. 6

 • Encadenar las promesas así: 6

 Ponte a Prueba 2 – Errores en el trabajo con promesas 6

 Código inicial (erróneo):..... 6

 Problema: 6

 Solución: 6

 Cómo visualizarlo 6

 Requisitos: 7

 • Editor de código: en mi caso, Visual Studio Code. 7

 • Navegador web actualizado: en mi caso, Google Chrome. 7

 • (Opcional) Node.js para ejecutar los scripts en consola. 7

 Pasos: 7

 1. Guardar los archivos .js y .html en la raíz del proyecto, muy importante. 7

 2. Abrir el archivo .html en el navegador o ejecutar el .js con Node..... 7

 3. Caso práctico 2: observar en consola cómo se ejecutan los temporizadores uno tras otro. .. 7

 4. Ponte a prueba 2: comprobar que el valor de la promesa solo aparece usando .then(). 7

5. Revisar la consola (F12 --> “Consola”) para verificar mensajes de prueba de autoría..... 7

Objetivos de la práctica..... 7

- Probar y comprender el código teórico del apartado 2..... 7
- Encadenar promesas para ejecutar tareas asíncronas en orden..... 7
- Detectar y corregir errores comunes al trabajar con promesas. 7
- Documentar correctamente cada ejercicio con nombre, fecha y capturas de pantalla. 7
- Presentar el proyecto en formato PDF junto con los archivos y pantallazos. 7

CONCLUSIÓN..... 7

INTRODUCCIÓN

En el ejercicio “Probad todo el código del Apartado 2” deberemos incluir en uno o varios fuentes (yo siempre recomiendo varios) absolutamente todo el código que aparece en la teoría de Programación Asíncrona, así como su ejecución y prueba de autoría.

En el ejercicio “Probad los Scripts del Apartado 2” deberemos ejecutar los scripts adjuntos y explicar de manera clara su funcionamiento. Recordad incluir pruebas de autoría.

En el “Caso práctico 2” realizaremos un programa que utilice promesas para ejecutar tres temporizadores en serie. Se mostrará un mensaje por consola cuando termine cada temporizador.

En el “Ponte a prueba 2” a través del código proporcionado deberemos investigar y explicar por qué no funciona bien y cómo podríamos arreglar dicho fragmento de código.

ENUNCIADO

Probad todo el código del Apartado 2

Incluid en uno o varios fuentes **todo** el código que aparece en la teoría de este apartado, así como su ejecución y prueba de autoría.

Probad los Scripts del Apartado 2

Ejecutad los scripts adjuntos y explicad su funcionamiento. Recordad incluir pruebas de autoría.

Caso práctico 2

Encadenamiento de promesas

Crea un programa que utilice **promesas** para ejecutar **tres temporizadores en serie**: cuando finalice el primer temporizador debe comenzar el segundo, y cuando finalice el segundo debe comenzar el tercero.

Se debe mostrar un **mensaje** por la consola cuando finalice **cada temporizador**. Además, se debe mostrar un **mensaje al finalizar**, independientemente de si se produce un error en algún temporizador.

Ponte a prueba 2

Errores en el trabajo con promesas

El código que se proporciona a continuación no funciona bien. ¿Por qué? ¿Cómo podrías arreglarlo?

```
function temporizador(tiempo) {  
  return new Promise(function(resolver, rechazar) {  
    setTimeout(function() {  
      resolver(`Temporizador de ${this.tiempo} ms  
terminado.`);  
    }, tiempo);  
  });  
}
```

```
let resultado = temporizador(3000);  
console.log(`El valor que devuelve el temporizador es:  
${resultado}.`);
```

Claves de resolución:

- Ejecuta el programa en NodeJS o en la consola del navegador. Observa qué aparece por pantalla.
- Analiza el código y observa qué valor devuelve la función **temporizador**.

DESARROLLO DE LA PRACTICA

Práctica DWEK – Unidad 4 Apartado 2

Descripción

Este apartado se centra en el uso de **promesas en JavaScript** para gestionar operaciones asíncronas de forma controlada.

El objetivo principal ha sido:

- Encadenar varias promesas para ejecutar temporizadores en serie (Caso práctico 2).
- Analizar y corregir errores muy comunes al trabajar con promesas y valores devueltos (Ponte a prueba 2).

Contenido del archivo

Código del Apartado 2 de teoría

Se ha probado absolutamente todo el código relacionado con:

- Creación de promesas mediante `new Promise()`.
- Resolución y rechazo de promesas con `resolve` y `reject`.
- Encadenamiento de tareas usando `.then()`.
- Manejo de errores con `.catch()`.
- Ejecución de código final con `.finally()`.

Cada archivo incluye comentarios con el nombre del autor y la fecha de realización.
Se han capturado los resultados por consola como prueba de ejecución.

Ejemplo de ejecución:

Salida por consola – DWEK_U04_A02_01.html

Párrafo 1

Párrafo 2

Párrafo 3

Autor: Mario Segura Abad

Fecha de realización: 11/01/2026

Salida por consola – DWEK_U04_A02_02.html

Título 1

Título 2

Autor: **Mario Segura Abad**

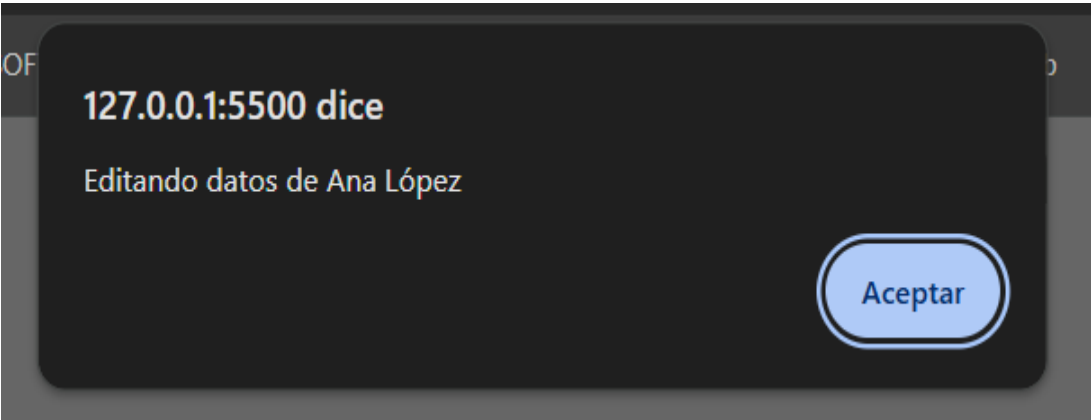
Fecha de realización: 11/01/2026

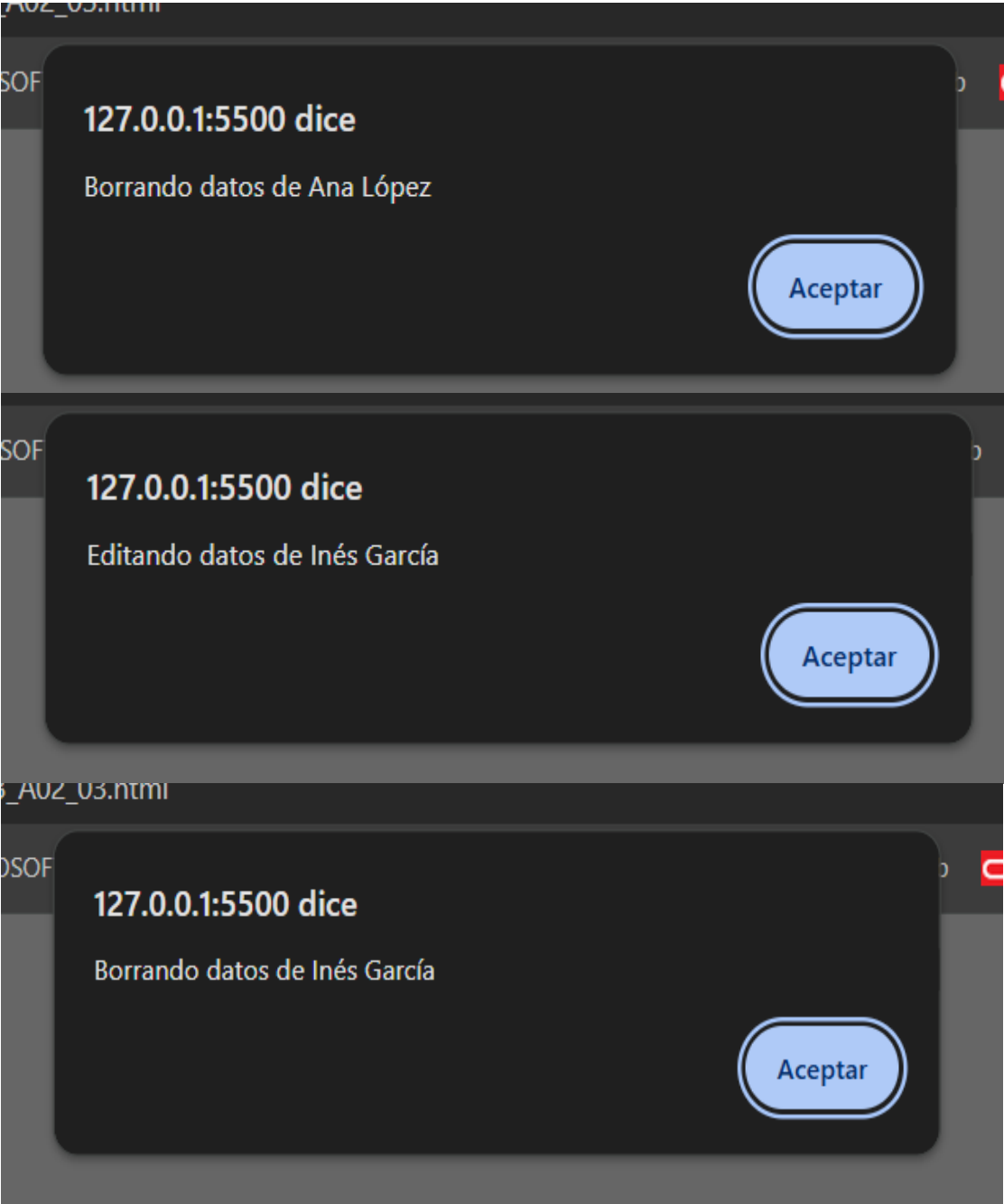
Salida por consola – DWEC_U04_A02_03.html

Ana López	Editar	Borrar
Inés García	Editar	Borrar

Autor: **Mario Segura Abad**

Fecha de realización: 11/01/2026





```

175
✖ ▶ Uncaught ReferenceError: funcionQueDevuelveUnaPromesa is VM414:9
not defined
    at <anonymous>:9:9

```

Salida por consola – DWEC_U04_A02_TEO_2.js

```
// Prueba de autoría
console.log("Autor: Mario Segura Abad");
console.log("Fecha de realización: 18/12/2025");
```

Autor: Mario Segura Abad [VM800:11](#)

Fecha de realización: 18/12/2025 [VM800:12](#)

⏪ undefined

Salida por consola – DWEC_U04_A02_TEO_3.js

```
// Prueba de autoría
console.log("Autor: Mario Segura Abad");
console.log("Fecha de realización: 18/12/2025");
```

Este mensaje se muestra antes que los mensajes de los temporizadores	VM1065:36
Autor: Mario Segura Abad	VM1065:39
Fecha de realización: 18/12/2025	VM1065:40
undefined	

Salida por consola – DWEC_U04_A02_TEO_4.js

```
// Prueba de Autoría
console.log("Autor: Mario Segura Abad");
console.log("Fecha de realización: 18/12/2025");
```

✖ ▶ Uncaught ReferenceError: miPromesa is not defined VM1316:6
at <anonymous>:6:12

Salida por consola – DWEC_U04_A02_TEO_5.js

```
// Prueba de autoría
console.log("Autor: Mario Segura Abad");
console.log("Fecha de realización: 08/01/2026");
❌ ▶ Uncaught SyntaxError: Identifier 't1' has already been declared VM1866:1
    at <anonymous>:1:1
```

Salida por consola – DWEC U04 A02 TEO 6.js

```
// Prueba de autoría
console.log("Autor: Mario Segura Abad");
console.log("Fecha de realización: 08/01/2026");
```

```
✖ ▶ Uncaught SyntaxError: Identifier 't1' has already been declared
    at <anonymous>:1:1
```

Salida por consola – DWEC_U04_A02_TEO_7.js

```
// Prueba de autoría
console.log("Autor: Mario Segura Abad");
console.log("Fecha de realización: 08/01/2026");
```

```
Autor: Mario Segura Abad
```

VM2348:40

```
Fecha de realización: 08/01/2026
```

VM2348:41

```
< undefined
```

```
✖ ▶ Uncaught Error: Error muy grave
    at <anonymous>:8:15
```

Caso Práctico 2 – Encadenamiento de promesas

Objetivo:

- Crear un programa que ejecute **tres temporizadores en serie** usando promesas.
- Mostrar un mensaje en consola al finalizar cada temporizador.
- Mostrar un mensaje final independiente de si hubo o no error.

Claves de resolución:

- Crear una función `temporizador(t)` que devuelva una promesa que se resuelve tras `t` milisegundos.
- Encadenar las promesas así:

```
temporizador(1000)
  .then(() => temporizador(2000))
  .then(() => temporizador(3000))
  .catch(error => console.error(error))
  .finally(() => console.log("Proceso finalizado."));
```

- Usar `.finally()` para mostrar el mensaje final pase lo que pase.

Este ejercicio nos ha permitido reforzar el uso de **promesas encadenadas** y la gestión de flujos asíncronos complejos.

Ejemplo de ejecución:

Salida por consola – CasoPractico2.js

```
// Prueba de autoría
console.log("Autor: Mario Segura Abad");
console.log("Fecha de realización: 08/01/2026");

Autor: Mario Segura Abad VM2596:41
Fecha de realización: 08/01/2026 VM2596:42
< undefined
Temporizador 1 finalizado VM2596:19
Temporizador 2 finalizado VM2596:24
Temporizador 3 finalizado VM2596:29
Proceso finalizado (con o sin errores) VM2596:37
```

Ponte a Prueba 2 – Errores en el trabajo con promesas

Código inicial (erróneo):

```
let resultado = temporizador(3000);
console.log(`El valor que devuelve el temporizador es:
${resultado}.`);
```

Problema:

- temporizador(3000) devuelve **una promesa**, no el resultado final.
- Por eso en consola aparece:
- “[object Promise]”
- El valor real solo está disponible cuando la promesa se resuelve.

Solución:

Usar `.then()` para acceder al resultado:

```
temporizador(3000).then(resultado => {
  console.log(resultado);
});
```

Este ejercicio nos ha reforzado la comprensión de **cómo funcionan las promesas**, cuándo se obtiene su valor y por qué no se puede acceder directamente a él.

Ejemplo de ejecución:

Salida por consola – PonteAPrueba2.js

```
// Prueba de autoría
console.log("Autor: Mario Segura Abad");
console.log("Fecha de realización: 08/01/2026");
```

Autor: Mario Segura Abad	VM2813:18
Fecha de realización: 08/01/2026	VM2813:19

⏪ undefined

El valor que devuelve el temporizador es: Temporizador de [VM2813:14](#)
3000 ms terminado.

Cómo visualizarlo

Requisitos:

- Editor de código: en mi caso, Visual Studio Code.
- Navegador web actualizado: en mi caso, Google Chrome.
- (Opcional) Node.js para ejecutar los scripts en consola.

Pasos:

1. Guardar los archivos .js y .html en la raíz del proyecto, muy importante.
2. Abrir el archivo .html en el navegador o ejecutar el .js con Node.
3. **Caso práctico 2:** observar en consola cómo se ejecutan los temporizadores uno tras otro.
4. **Ponte a prueba 2:** comprobar que el valor de la promesa solo aparece usando .then().
5. Revisar la consola (F12 --> "Consola") para verificar mensajes de prueba de autoría.

Objetivos de la práctica

- Probar y comprender el código teórico del apartado 2.
- Encadenar promesas para ejecutar tareas asíncronas en orden.
- Detectar y corregir errores comunes al trabajar con promesas.
- Documentar correctamente cada ejercicio con nombre, fecha y capturas de pantalla.
- Presentar el proyecto en formato PDF junto con los archivos y pantallazos.

CONCLUSIÓN

Esta práctica nos ha sido muy útil para consolidar el uso de **promesas en JavaScript** para controlar procesos asíncronos.

Se ha comprobado cómo encadenar tareas, cómo manejar errores y cómo interpretar correctamente el valor devuelto por una promesa.

La práctica también nos ha sido muy útil para intentar entender la programación asíncrona moderna.